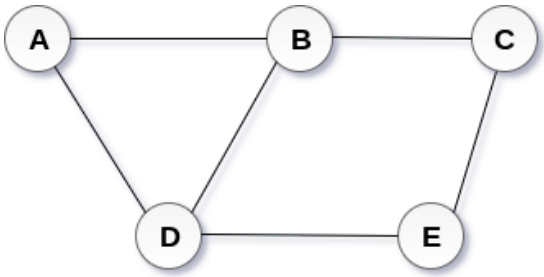
Graphs

A graph can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as **a cyclic tree**, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship.

Definition

A graph G can be defined as an ordered set G (V, E) where V(G) represents the set of vertices and E(G) represents the set of edges which are used to connect these vertices. A Graph G (V, E) with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.

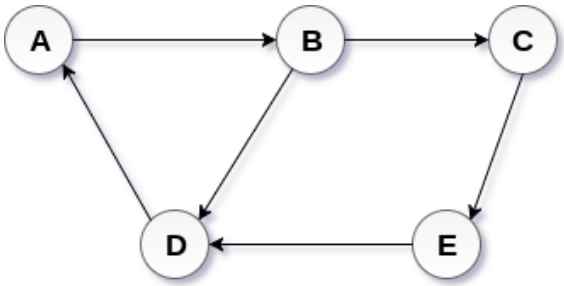


Undirected Graph

Directed and Undirected Graph

A graph can be directed or undirected. However, in an undirected graph, edges are not associated with the directions with them. An undirected graph is shown in the above figure since its edges are not attached with any of the directions. If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B.

In a directed graph, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. Node A is called initial node while node B is called terminal node. A directed graph is shown in the following figure.



Directed Graph

Graph Terminology

Path: A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.

Closed Path: A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if $V_0=V_N$

Simple Path: If all the nodes of the graph are distinct with an exception $V_0=V_N$, then such path P is called as closed simple path.

Cycle: A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

Connected Graph: A connected graph is the one in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.

Complete Graph: A complete graph is the one in which every node is connected with all other nodes. A complete graph contain n(n-1)/2 edges where n is the number of nodes in the graph.

Weighted Graph: In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as w(e) which must be a positive (+) value indicating the cost of traversing the edge.

Digraph: A digraph is a directed graph in which each edge of the graph is associated with some direction and the traversing can be done only in the specified direction.

Loop: An edge that is associated with the similar end points can be called as Loop.

Adjacent Nodes: If two nodes u and v are connected via an edge e, then the nodes u and v are called as neighbors or adjacent nodes.

Degree of the Node: A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.

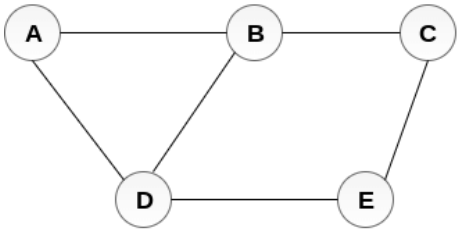
Graph Representation

By Graph representation, we simply mean the technique which is to be used in order to store some graph into the computer's memory. There are two ways to store Graph into the computer's memory. In this part of this tutorial, we discuss each one of them in detail.

1. Sequential Representation (Adjacency Matrix)

In sequential representation, we use adjacency matrix to store the mapping represented by vertices and edges. In adjacency matrix, the rows and columns are represented by the graph vertices. A graph having n vertices will have a dimension **n x n**. An entry M_{ij} in the adjacency matrix representation of an undirected graph G will be 1 if there exists an edge between V_i and V_j.

An undirected graph and its adjacency matrix representation are shown in the following figure.



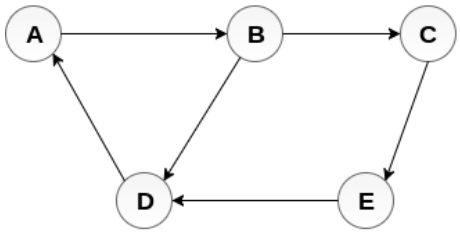
Undirected Graph

$$\begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 0 & 1 & 0 \\ B & 1 & 0 & 1 & 1 & 0 \\ C & 0 & 1 & 0 & 0 & 1 \\ D & 1 & 1 & 0 & 0 & 1 \\ E & 0 & 0 & 1 & 1 & 0 \end{array}$$

Adjacency Matrix

In the above figure, we can see the mapping among the vertices (A, B, C, D, E) is represented by using the adjacency matrix which is also shown in the figure. There exist different adjacency matrices for the directed and undirected graph. In directed graph, an entry A_{ij} will be 1 only when there is an edge directed from V_i to V_j.

A directed graph and its adjacency matrix representation are shown in the following figure.



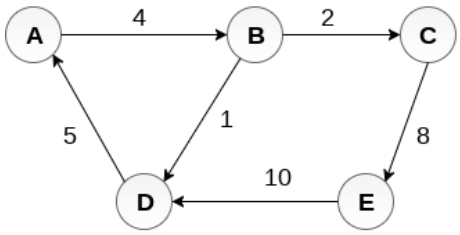
Directed Graph

$$\begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 0 & 0 & 0 \\ B & 0 & 0 & 1 & 1 & 0 \\ C & 0 & 0 & 0 & 0 & 1 \\ D & 1 & 0 & 0 & 0 & 0 \\ E & 0 & 0 & 0 & 1 & 0 \end{array}$$

Adjacency Matrix

Representation of weighted directed graph is different. Instead of filling the entry by 1, the Non-zero entries of the adjacency matrix are represented by the weight of respective edges.

The weighted directed graph along with the adjacency matrix representation is shown in the following figure.



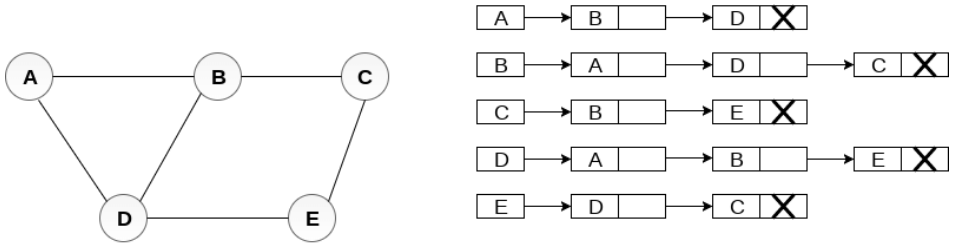
Weighted Directed Graph

$$\begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 4 & 0 & 0 & 0 \\ B & 0 & 0 & 2 & 1 & 0 \\ C & 0 & 0 & 0 & 0 & 8 \\ D & 5 & 0 & 0 & 0 & 0 \\ E & 0 & 0 & 0 & 10 & 0 \end{array}$$

Adjacency Matrix

2. Linked Representation (Adjacency List)

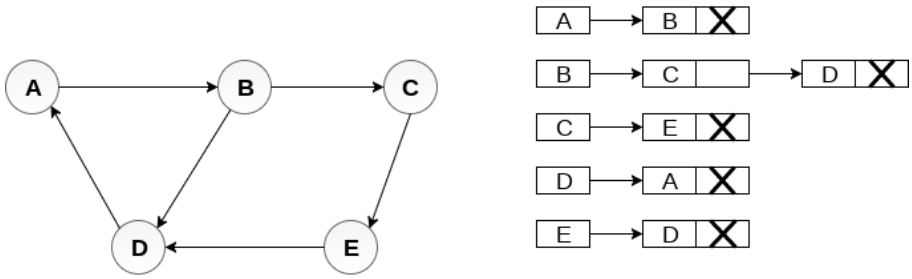
In the linked representation, an adjacency list is used to store the Graph into the computer's memory. Consider the undirected graph shown in the following figure and check the adjacency list representation.



Undirected Graph **Adjacency List**

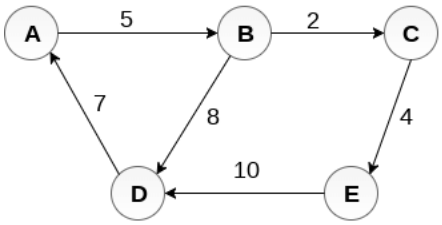
An adjacency list is maintained for each node present in the graph which stores the node value and a pointer to the next adjacent node to the respective node. If all the adjacent nodes are traversed, then store the NULL in the pointer field of last node of the list. The sum of the lengths of adjacency lists is equal to the twice of the number of edges present in an undirected graph.

Consider the directed graph shown in the following figure and check the adjacency list representation of the graph.

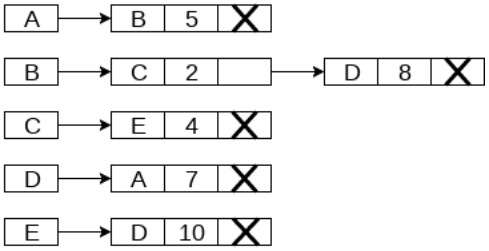


Directed Graph **Adjacency List**

In a directed graph, the sum of lengths of all the adjacency lists is equal to the number of edges present in the graph.
In the case of weighted directed graph, each node contains an extra field that is called the weight of the node. The adjacency list representation of a directed graph is shown in the following figure.



Weighted Directed Graph



Adjacency List

Graph Traversal Algorithms

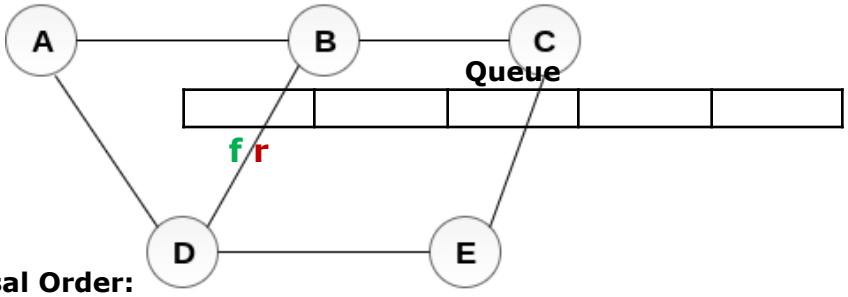
In this part of the tutorial we will discuss the techniques by using which, we can traverse all the vertices of the graph. Traversing the graph means examining all the nodes and vertices of the graph. There are two standard methods by using which, we can traverse the graphs. Let's discuss each one of them in detail.

- Breadth First Search
- Depth First Search

Breadth First Search (BFS) Algorithm

Breadth first search is a graph traversal algorithm that starts traversing the graph from root node and explores all the neighboring nodes. Then, it selects the nearest node and explore all the unexplored nodes. The algorithm follows the same process for each of the nearest node until it finds the goal.

The algorithm of breadth first search is given below. The algorithm starts with examining the node A and all of its neighbors. In the next step, the neighbors of the nearest node of A are explored and process continues in the further steps. The algorithm explores all neighbors of all the nodes and ensures that each node is visited exactly once and no node is visited twice.



Traversal Order:

Visited	
A	1
B	1
C	1
D	1
E	1

Program for BFS

```
#include<stdio.h>
int a[20][20], q[20], visited[20], n, i, j, f = 0, r = -1;
void bfs(int v)
{
	for(i = 1; i <= n; i++)
	{
		if(a[v][i] && !visited[i])
			q[++r] = i;
		if(f <= r)
		{
			visited[q[f]] = 1;
			bfs(q[f++]);
		}
	}
}

int main()
{
	int v;
	printf("\n Enter the number of vertices:");
	scanf("%d", &n);
	for(i=1; i <= n; i++)
	{
		visited[i] = 0;
	}
	printf("\n Enter graph data in matrix form:\n");
	for(i=1; i<=n; i++)
	{
		for(j=1;j<=n;j++)
		{
			scanf("%d", &a[i][j]);
		}
	}
	printf("\n Enter the starting vertex:");
	scanf("%d", &v);
	bfs(v);
	printf("\n The node which are reachable are:\n");
	for(i=1; i <= n; i++)
	{
		if(visited[i])
			printf("%d\t", i);
		else
		{
			printf("\n BFS is not possible. Not all nodes are reachable");
			break;
		}
	}
	return 0;
}
```

	1	**2**	**3**	**4**
1	0	1	1	0
2	1	0	1	0
3	0	1	1	0
4	1	0	0	1

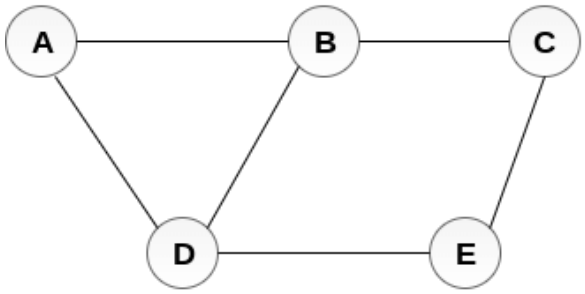
Depth First Search (DFS) Algorithm

Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

The data structure which is being used in DFS is stack. The process is similar to BFS algorithm. In DFS, the edges that lead to an unvisited node are called discovery edges while the edges that lead to an already visited node are called block edges.

Example

Consider the graph G along with its adjacency list, given in the figure below. Calculate the order to print all the nodes of the graph starting from node H, by using depth first search (DFS) algorithm.



Visited	
A	1
B	1
C	1
D	1
E	1

top

Stack

Traversal Order:

Program for DFS

	1	2	3	4
1	0	1	1	0
2	1	0	1	0
3	0	1	1	0
4	1	0	0	0

```
#include<stdio.h>
int a[20][20], visited[20], n;
void dfs(int v)
{
	int i;
	visited[v]  = 1;
	for(i=1; i<=n; i++)
	{
		if(a[v][i] && !visited[i])
		{
			printf("\n %d -> %d", v, i);
			dfs(i);
		}
	}
}

int main()
{
	int i, j, count=0;
	printf("\n Enter number of vertices:");
	scanf("%d", &n);
	for(i=0; i<n; i++)
	{
		visited[i] = 0;
	}
	printf("n Enter the adjacency matrix: n");
	for(i=1; i<=n; i++)
		for(j=1; j<=n; j++)
			scanf("%d", &a[i][j]);
	dfs(1);
	printf("\n");
	for(i=0; i<n;  i++)
	{
		if(visited[i])
		count++;
	}
	if(count==n-1)
		printf("\n Graph is connected");
	else
		printf("\n Graph is not connected");
	return 0;
}
```